PARCIAL DISEÑO DE SOFTWARE 2023

TEORÍA

Seleccionar las respuestas correctas

Existen diferentes grados de Grados de cohesión, cuales son grados de cohesión correctas:

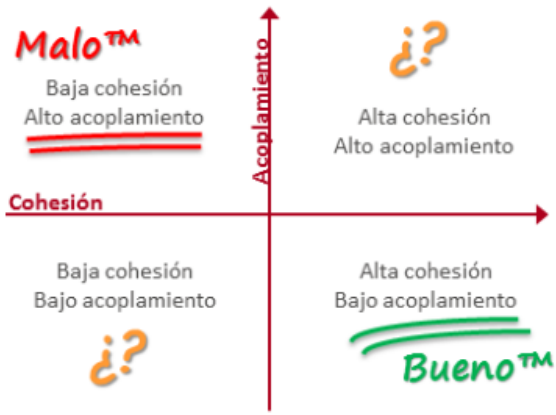
- Cohesión Coincidentes
- Cohesión Lógica
- Cohesión Temporal
- Cohesión Procedural
- Cohesión de Datos
- Cohesión de Procedimiento
- Cohesión de Información
- Cohesión Funcional

FACTORES DE CALIDAD: COHESION

Se define como la medida de fuerza o relación funcional existente entre las sentencias o grupos de sentencias de un mismo módulo. Un módulo cohesionado ejecutará una única tarea sencilla interactuando muy poco o nada con el resto de módulos del programa. Se persigue que los módulos tengan una alta cohesión.

En el ***diseño estructurado*** podemos encontrarnos con los siguientes siete tipos de cohesión (de la mejor o más deseable a la menos recomendable):

Cohesión funcional
Cohesión secuencial
Cohesión comunicacional
Cohesión procedimental
Cohesión temporal
Cohesión lógica
Cohesión casual o **coincidental**:



Cohesión funcional: Los elementos del módulo están relacionados en el desarrollo de una única función.

Cohesión secuencial: Un módulo realiza distintas tareas en secuencia, de forma que las entradas de cada tarea son las salidas de la tarea anterior. No es una mala cohesión si las tareas implicadas no son muy complejas y requieren pocas líneas de código.

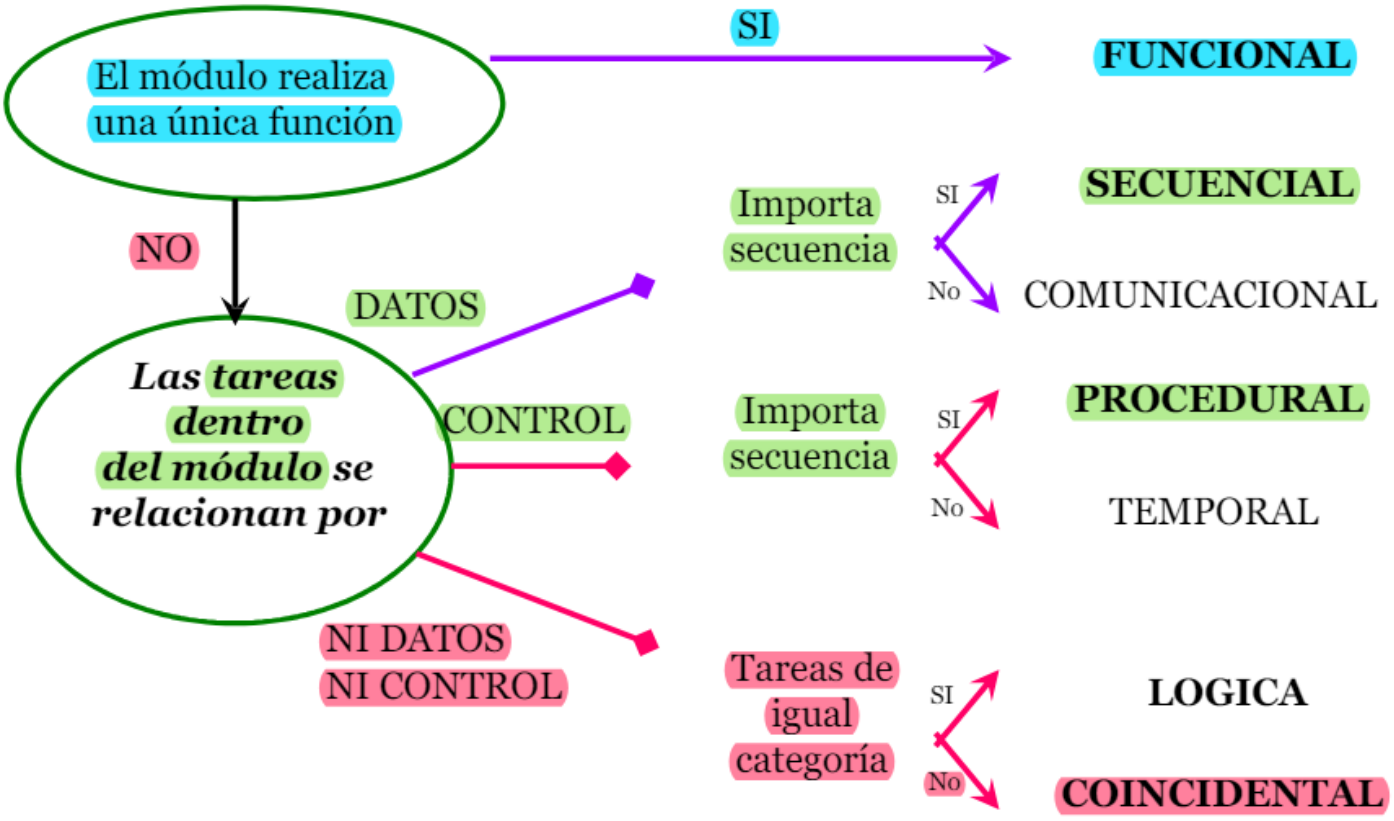
Cohesión comunicacional: El módulo realiza actividades paralelas usando los mismos datos de entrada y salida. Como en el caso anterior, tampoco se trata de un mal tipo de cohesión si las tareas son relativamente sencillas.

Cohesión procedimental: El módulo tiene una serie de funciones relacionadas por un procedimiento efectuado por el código (a modo de biblioteca). Es similar a la secuencial, pero puede incluir el paso de controles. Será deseable que las funciones estén relacionadas o realicen tareas dentro del mismo ámbito (p.e. la biblioteca string.h de C contiene funciones para operar con cadenas de caracteres).

Cohesión temporal: Los elementos del módulo están implicados en actividades relacionadas con el tiempo.

Cohesión lógica: Las actividades que realiza el módulo tienen la misma categoría. Esto es: es como si se tuvieran partes independientes dentro del mismo módulo.

Cohesión casual o coincidental: Los elementos del módulo contribuyen a las actividades relacionándose mutuamente de una manera poco significativa. Este tipo de cohesión viola el principio de independencia y de caja negra de los módulos..



Grados de cohesión

- **Muy baja Cohesión**: una **clase** es la **única responsable** de muchas cosas distintas en áreas funcionales diferentes (se deben dividir las responsabilidades en una familia de clases relacionadas).
- **Baja Cohesión**: una **clase** tiene la **responsabilidad exclusiva** de una **tarea compleja** dentro de una área funcional (dividir la clase en una familia de clases ligeras que compartan el trabajo).
- **Alta Cohesión**: una **clase tiene responsabilidades moderadas** en un área y colabora con las otras para realizar las tareas (supuesto ideal)
- **Cohesión Coincidentes**: Módulos múltiples tareas.
- **Cohesión Lógica:** Módulo con muchas tareas, pero en ejecución solo se ejecuta una.
- **Cohesión de Procedimiento:** Módulo con muchas secuencias de pasos.
- **Cohesión de Información:** Trabajan sobre los mismos datos.
- **Cohesión Funcional:** Módulo que ejecuta una única tarea.

Complete las afirmaciones

GRASP

- **POLIMORFISMO** es un principio de GRASP que enfatiza la importancia de utilizar interfaces y clases abstractas para permitir que los objetos se comporten de múltiples maneras.

POLIMORFISMO

- Tiene varios significativos, en este contexto significa **Asignar el mismo Nombre a servicios diferentes**
- En POO, el polimorfismo se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos

3 FORMAS DE POLIMORFISMO

- **Sobre-escritura de métodos** : Métodos de una subclase sobrescriben los métodos de la superclase
- **Sobre-escritura de métodos (impl) abstractos**: Métodos de la subclase implementan los métodos abstractos de la superclase abstracta (clase abstracta)
- **Sobre-escritura de métodos (impl) a través de interface**: Métodos de una clase concreta implementan los métodos de la interface

- **INDIRECCIONES** GRASP propone que se debe utilizar una capa intermedia para reducir las dependencias directas entre clases de altos nivel y clases de bajo nivel.

INDIRECCIONES

A veces se necesita evitar el acoplamiento directo entre dos objetos, para desacoplar los objetos se utiliza un intermediario (crea una indirección entre los objetos)

Ejemplo: La clase pago tiene que comunicar a otro sistema de “impuesto” que el pago se realizó, se podría añadir una clase como un Adaptador que es una clase cuyo objetivo es ser intermediario.

- El principio de **DISEÑO MODULAR** en GRASP sugiere que un sistema debe dividirse en módulos o componentes independientes, cada uno con una única responsabilidad.

DISEÑO MODULAR

La modularidad es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos COHESIVOS y débilmente ACOPLADOS.

GOF

- En el patrón de diseño **COMPOSITE**, los elementos individuales y las composiciones de elementos se tratan de la misma manera a través de una interfaz común.

COMPOSITE (COMPUESTO)

Sirve para construir objetos complejos a partir de otros más simples y similares entre sí, gracias a la composición recursiva y a una estructura en forma de árbol

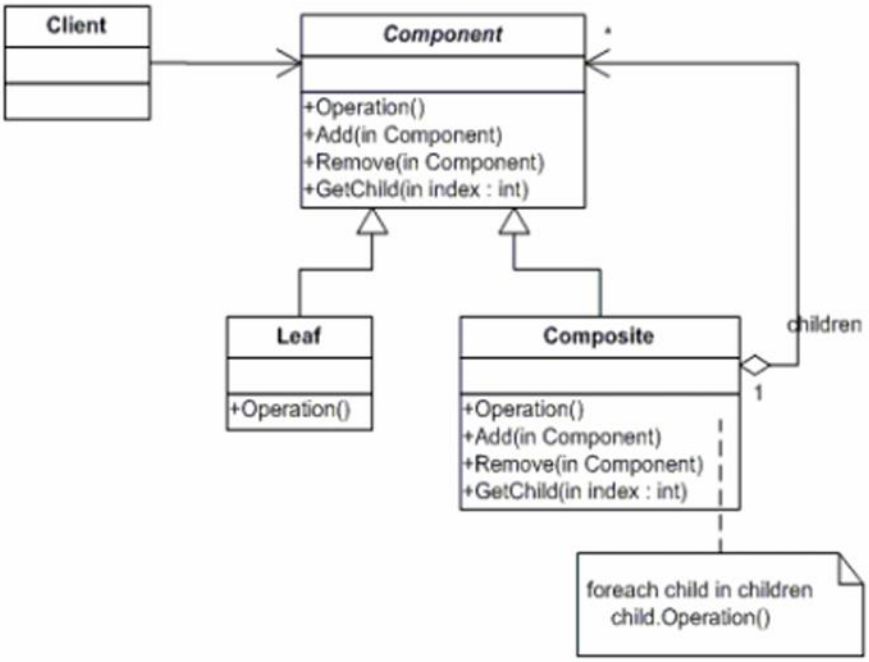
- *Compone objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.*

Aplicabilidad:

- Permite representar jerarquías de objetos, similar a una estructura de árbol.
- Permite tratamiento uniforme de objetos simples y complejos, así como composiciones recursivas.
- Simplifica el código de los clientes, que sólo usan una interfaz
- Facilita añadir nuevos componentes sin afectar a los clientes.
- Cuando se quiera que los clientes puedan ignorar la diferencia entre objetos simples y compuestos y tratarlos uniformemente.

Se debe utilizar este patrón cuando:

- Se busca representar una jerarquía de objetos como “parte-todo”
- Se busca que el cliente pueda ignorar la diferencia entre objetos primitivos y compuestos (para que pueda tratarlos de la misma manera)



Component: Declara la interfaz de los objetos involucrados en la composición, declara una interfaz para acceder y manejar sus hijos

Leaf: Representa objetos hoja en la composición

Composite: Define el comportamiento de los componentes que tienen hijos, almacena los componentes hijo, implementa operaciones relacionadas con los hijos

Client: Manipula los objetos de la composición a través de Component

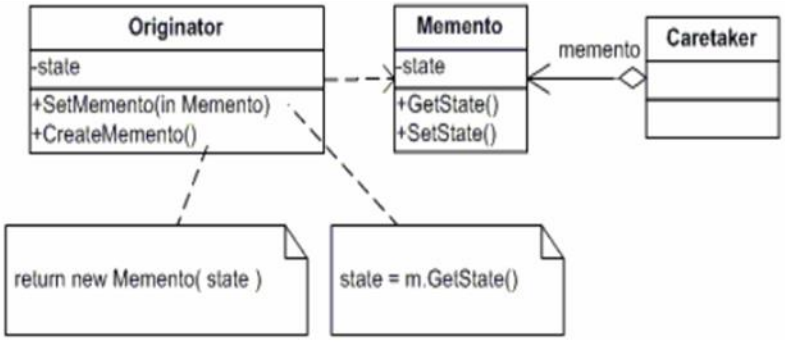
- El patrón de diseño **MEMENTO**, se utiliza para capturar y externalizar un estado interno de un objeto, de modo que este pueda ser restaurado en el futuro.

MEMENTO (RECUERDO)

Representa y externaliza el estado interno de un objeto sin violar la encapsulación, de forma que éste puede volver a dicho estado más tarde.

Se usa cuando:

- Se necesita restaurar el sistema desde estados pasados.
- Se quiere facilitar el hacer y deshacer de determinadas operaciones, para lo que habrá que guardar los estados anteriores de los objetos sobre los que se opere (o bien recordar los cambios de forma incremental)



Memento: Almacena el estado interno del objeto Originator, protege del acceso por parte de objetos distintos al Originator

Originator: Crea un objeto memento, que contiene una foto fija de su estado interno

Caretaker: Es el responsable de mantener la seguridad del objeto memento, no opera o examina el contenido de memento

- El patrón de diseño **VISITOR** se utiliza para representar una operación a ser realizada en los elementos de una estructura de objetos.

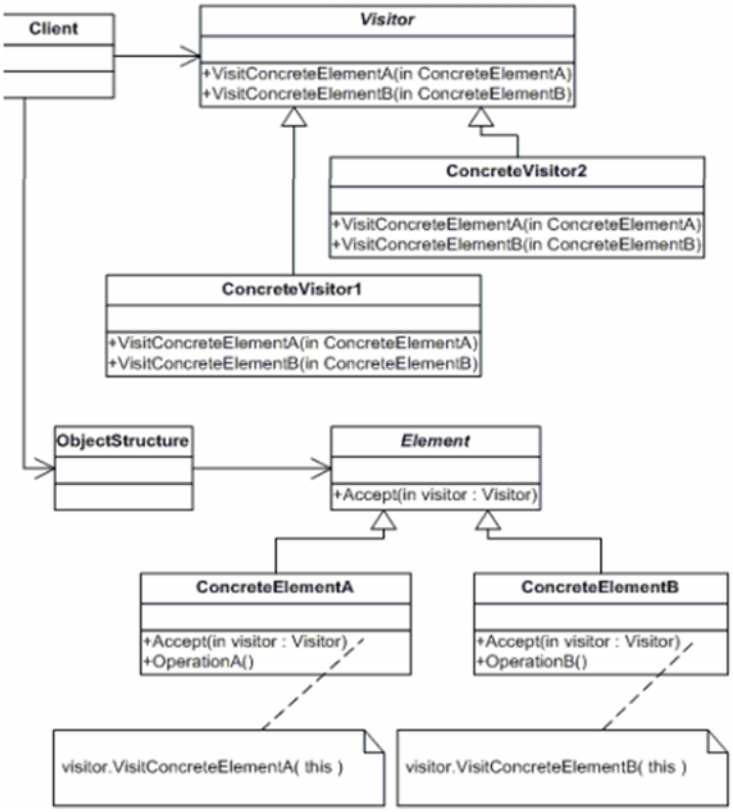
VISITOR (VISITANTE)

Es una forma de separar el algoritmo de la estructura de un objeto.

- *Representa una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.*

Este patrón debe utilizarse cuando:

- Una estructura de objetos contiene muchas clases de objetos con distintas interfaces y se desea llevar a cabo operaciones sobre estos objetos que son distintas en cada clase concreta.
- Se quiere llevar a cabo muchas operaciones dispares sobre objetos de una estructura de objetos sin tener que incluir dichas operaciones en las clases.



Visitor: Establece las operaciones a realizar
ConcreteVisitor: Implementa dichas operaciones
Element: Define un método Accept() que toma un Visitor como argumento
ConcreteElement: Implementa el método Accept()
ObjectStructure:

RUP

1. Según RUP, ¿Cuál es la relación entre el modelo de casos de uso y el modelo de diseño?

RESPUESTA CHATGPT

El modelo de casos de uso define qué debe hacer el sistema, mientras que el modelo de diseño especifica cómo el sistema realizará esas funcionalidades. Esta relación asegura que el desarrollo del software esté alineado con los requisitos del usuario y que cada funcionalidad esté adecuadamente implementada y testeada.

2. ¿Cuál de las siguientes afirmaciones, no es un objetivo primordial de la fase de elaboración? Justifique

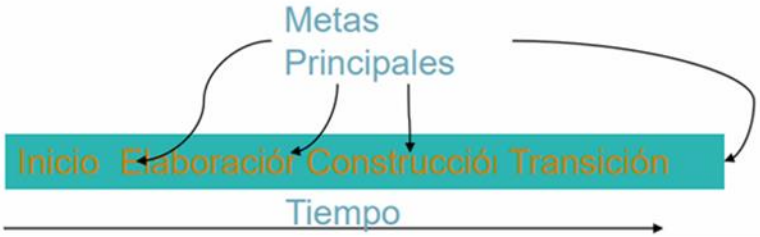
- Definición, validación y línea de base de la arquitectura tan rápidamente como sea posible.
- La eliminación de los elementos de mayor riesgo del proyecto.
- Crear una primera versión del modelo de negocio.
- Línea Base de un plan de alta fidelidad para la fase de construcción.
- Demostrar que la arquitectura de referencia respaldara la visión del producto a un costo razonable en un tiempo razonable.

El modelo de negocio (caso de negocio en el apunte) se realiza en la etapa de Inicio.

FASES EN RUP

FASES	OBJETIVOS
Inicio	Entender el problema
Elaboración	Entender la solución
Construcción	Tener la solución
Transición	Aceptar la solución

Inicio: Define el objetivo y alcance del proyecto, determinar los CU preliminares, estimación, costo, identificar riesgos
Elaboración: Plan del proyecto, especificación de características, arquitectura base, CU
Construcción: Construir el producto
Transición: Transición del producto a la comunidad del usuario



FASE DE ELABORACIÓN

- **PROPÓSITO**
  - Analizar el dominio del problema
  - Establecer una buena arquitectura
  - Lidiar con los elementos de riesgo más altos del proyecto
  - Desarrollar un plan comprensivo mostrando cómo el proyecto será completado
- **RESULTADO**
  - Un modelo del dominio y de casos de uso 80% completo
  - Requerimientos suplementarios que capturen los requerimientos no funcionales y cualesquiera requerimientos que no estén asociados con un caso de uso específico
  - Una lista de riesgos revisada

PRACTICA

Reserva Ya es una solución con que se pretende agilizar el proceso de armado de paquetes turísticos para vendedores, el sistema debe permitir la organización, elaboración y publicidad de productos turísticos.

Los vendedores pueden definir el producto a vender, básicamente, los productos a vender son los siguientes:

- **Paquetes Predefinidos**
- **Paquetes Configurables**

Los Paquetes Predefinidos se dividen en diferentes categorías:

- **Urbano**: Desarrollado en ciudades principalmente en aquellas que son Patrimonio de la Humanidad.
- **Monumental**: Vinculado exclusivamente a monumentos histórico-artísticos que pueden estar alejados de núcleos de población importantes.
- **Arqueológico**: Vinculado a yacimientos y sitios arqueológicos que pueden estar alejados de núcleos de población importantes.
- **Aventura**: El turismo de aventura tiene como objetivo principal el fomento de las actividades de aventura en la naturaleza.

Los Paquetes Configurables, los arman los vendedores de acuerdo con los gustos del cliente. Los paquetes configurables se arman seleccionando medio de transporte, alojamiento, actividades, etc., por ese motivo el sistema debe contener información de diferentes proveedores de estos servicios:

Proveedores de transporte:

- Transporte aéreo (líneas regulares y chárter), trabajan con aerolíneas regulares, aunque en muchas ocasiones lo hacen con compañías de vuelos chárter.
- Transporte terrestre (autocar, tren, Automóvil y alquileres de automóviles, ejemplo Hertz)
- Transporte marítimo (líneas regulares y cruceros)

Proveedores de servicio de alojamiento:

El sistema debe tener poder seleccionar la zona que el cliente desea visitar y luego encontrar el servicio de alojamiento más acorde con las preferencias y costo que el cliente solicito: ejemplo:

- Hoteles contemplando las diferentes categorías 2, 3, 4, 5 *
- apart (1,2,3,4 llaves)
- bungalow
- campings

Régimen: Desayuno, media pensión, all inclusive

Actividades: En cada ciudad a visitar, tiene un listado actividades a realizar. Ejemplo: Visitas cruceros, etc.

Proveedores de actividades:

El sistema debe tener poder seleccionar la zona que el cliente desea visitar y luego encontrar el servicio de actividades

- Senderismo
- Turismo deportivo
- Cicloturismo

- Paseos nocturnos
- Paseos de ciudad
- Paseo de puntos turísticos
- Rutas ecológicas
- Montañismo

Cada actividad maneja: Precios, días, horarios, tiempo de actividad, etc.

Proveedores de servicio de cobertura Medica

El producto por definir sea un paquete predefinido o no, debe contener la siguiente información:

- Origen o lugar de partida, fecha
- Destino de viaje, fecha
- Lugares intermedios, fecha
- Servicio de alojamiento, por lugar y fecha
- Servicio de transporte
- Transporte (alquiler de auto)
- Actividades
- Servicio de alimentación: Se puede incluir alimentos y bebidas (Sin pensión, media pensión o All Inclusive)
- Cobertura médica, internacional (si el viaje es al exterior)
- Cantidad de viajeros: TipoDoc y Numero, Nombre y Apellido, Edad, Nacionalidad.
- Precio: Cuando se viaja en familia se obtienen precios preferenciales.

Una vez definido por parte del vendedor el Producto a vender, de acuerdo a las preferencias del cliente, se define la forma de pago (TC, TD, transferencias, etc.) una vez finalizando, el vendedor puede cerrar el paquete.

El primer día de trabajo, el programador comenzó a definir las clases, y la primera versión, fue la siguiente:



Se Pide:

1. Hacer el Diagrama de Clases **completo**
2. Diagrama de Secuencia, con el escenario de armado de paquete configurable
3. Diagrama de Despliegue y Diagrama de Componentes